

KEEP CODING BOOTCAMP

Proyecto final

Radiografía del mercado laboral de inteligencia artificial y datos

Memoria técnica del proyecto

Este documento incluye: análisis exploratorio, limpieza de datos, modelo predictivo de salario y despliegue mediante API.

Equipo del proyecto

Beatriz Peña

Isabel García-Tenorio

Nur Younis

Septiembre de 2026

Índice

Introducción y objetivo del proyecto

PARTE I - Exploración, calidad y limpieza de datos

1. Carga de datos
2. Descripción del dataset
3. Relación entre tablas
4. Calidad de datos
5. Limpieza de datos
6. Análisis por tipo de variable
7. Resumen de la fase de exploración y limpieza

PARTE II -Modelado predictivo, NLP, Spark y despliegue

8. Correlación
9. Cruce del salario con el índice de coste de vida por estado (EE. UU.)

10. Extracción de habilidades técnicas desde el texto (NLP)
11. Preparar la tabla para el modelo de salario
12. Entrenar el modelo de salario (Random Forest)
13. Mejora del modelo: logaritmo, Gradient Boosting y experiencia
14. ETL y verificación de métricas con Apache Spark
15. Arquitectura de despliegue: API
16. Conclusiones generales del proyecto
17. Uso de IA

Introducción y objetivo del proyecto

Este proyecto responde a una pregunta muy concreta: dado el perfil de una oferta de empleo en el ámbito de datos e inteligencia artificial (las habilidades que pide, la industria a la que pertenece y la zona geográfica en la que se ubica), ¿es posible estimar cuánto va a pagar? Para responderla, el equipo ha trabajado sobre el dataset público LinkedIn Job Postings, que reúne más de 120.000 ofertas de empleo reales junto con información asociada sobre las empresas, los salarios, las habilidades solicitadas y los beneficios ofrecidos.

El trabajo se ha organizado en dos grandes bloques que se complementan y sobre los que se estructura esta memoria. El primero, cubre la carga, exploración, evaluación de calidad y limpieza del conjunto de datos: comprender qué información hay disponible, detectar problemas (nulos, duplicados, inconsistencias, valores atípicos) y tomar decisiones justificadas sobre cómo tratarlos, dejando el dataset en condiciones de ser analizado con garantías. El segundo, parte de ese trabajo ya realizado para construir un pipeline completo de ciencia de datos: análisis de correlaciones, incorporación de una fuente de datos externa (índice de coste de vida por estado), extracción de habilidades técnicas mediante procesamiento de lenguaje natural, entrenamiento y evaluación de un modelo de Machine Learning para predecir el salario, comparación con algoritmos alternativos, verificación cruzada de resultados con Apache Spark y despliegue del modelo como un servicio real mediante una API con FastAPI.

A lo largo de la memoria se ha priorizado la metodología por encima de mostrar únicamente los resultados favorables: se documentan tanto las decisiones que funcionaron como las que no, y se explica el razonamiento detrás de cada una. Un ejemplo central de este proyecto -el índice de coste de vida, que resulta ser la variable

individual más importante del modelo a pesar de tener una correlación directa con el salario prácticamente nula-ilustra precisamente esta filosofía: no se trata de aplicar un concepto, sino de entender de verdad los datos con los que se trabaja.

El resultado final es un pipeline funcional y verificado de principio a fin: desde datos crudos, sucios e incompletos, tal y como se presentan en cualquier problema real, hasta un servicio capaz de predecir un salario a través de una petición HTTP, pasando por decisiones de limpieza documentadas, un modelo entrenado y evaluado con rigor, y una validación cruzada de resultados mediante una herramienta de procesamiento distribuido (Spark).

PARTE I

Exploración, calidad y limpieza de datos

Esta primera parte de la memoria recoge el trabajo de exploración, evaluación de calidad y limpieza del dataset. Es el punto de partida indispensable sobre el que se apoya todo el modelado posterior: sin comprender qué contienen realmente los datos y sin tomar decisiones razonadas sobre sus problemas, cualquier modelo construido encima sería poco fiable.

1. Carga de datos

El proyecto parte de un conjunto de datos relacional compuesto por once tablas, centradas en tres grandes bloques de información: las ofertas de empleo (postings), las empresas que las publican (companies y sus tablas asociadas) y los datos salariales y de habilidades vinculados a cada oferta. La tabla principal, postings, contiene 123.849 ofertas de empleo, sobre la que giran el resto de tablas mediante relaciones de clave foránea.

2. Descripción del dataset

Antes de analizar cualquier relación entre variables, se realizó un inventario detallado de cada una de las once tablas: nombre, tipo de dato, papel que desempeña (identificador, descriptiva, temporal, geográfica, económica...), número y porcentaje de valores nulos, y número de valores únicos. Este inventario sistemático permitió detectar de forma

temprana qué variables presentaban problemas de calidad relevantes y cuáles podían utilizarse con garantías.

2.1 Tabla principal: postings (123.849 filas)

Es la tabla más compleja del dataset, con variables muy dispares en cuanto a calidad. Destacan especialmente los problemas de nulos en las variables económicas y algunas geográficas y temporales:

Variable	% nulos	Únicos	Observación
max_salary / min_salary	75,94%	5.321 / 4.612	Solo una minoría de ofertas declara salario directamente
med_salary	94,93%	1.417	Prácticamente inutilizable por sí sola
pay_period / currency / compensation_type	70,87%	5 / 6 / 1	Ligado a la disponibilidad de salario
applies	81,17%	274	Número de solicitudes; muy incompleta
remote_allowed	87,69%	1 valor observado	Solo aparece el valor 1; no permite inferir "no remoto"
formatted_experience_level	23,75%	6	Nivel de experiencia; nulos tratados como categoría propia
skills_desc	98,03%	2.212	Campo de texto libre prácticamente vacío
closed_time	99,13%	698	Casi siempre nula
title / description	0% / 0,006%	72.521 / 107.827	Variables de texto, buena calidad

Variable	% nulos	Únicos	Observación
views	1,36%	684	Buena calidad, aunque de escasa relación con el salario

Tabla 1. Resumen de calidad de las variables más relevantes de postings.

2.2 Tablas complementarias

El resto de tablas aporta información sobre empresas, industrias, habilidades y beneficios, en general con mejor calidad de datos:

- companies (24.473 registros): datos identificativos y descriptivos de la empresa, con company_size incompleto (11,33% de nulos, 7 categorías) y el campo state con inconsistencias de formato relevantes.
- employee_counts (24.473 registros): número de empleados y de seguidores por empresa, sin nulos.
- industries / company_industries / jobs_industries: catálogo de 422 industrias y sus relaciones con empresas y ofertas; industry_name tiene un 8,06% de nulos.
- skills / jobs_skills: catálogo de 35 habilidades y su asociación a las ofertas (213.768 asociaciones en total).
- salaries (40.785 registros): información salarial detallada por periodicidad, con valores nulos elevados en med_salary (83,23%) pero buena cobertura en min_salary y max_salary.
- benefits (67.943 registros): tipo de beneficio asociado a cada oferta (12 categorías).

3. Relación entre tablas

El modelo relacional se organiza en tres grupos de tablas: las tablas principales (postings, companies, salaries), que concentran la información central del análisis; las tablas de referencia o catálogo (skills, industries), que normalizan valores repetidos; y las tablas de relación (job_skills, job_industries, company_industries, company_specialities), que conectan las tablas principales con los catálogos mediante claves foráneas. A ellas se suman dos tablas auxiliares (benefits y employee_counts) con información complementaria sobre cada oferta y cada empresa respectivamente.

Esta estructura relacional se verificó de forma sistemática en la fase de calidad de datos (apartado 4.5), comprobando que las claves foráneas declaradas realmente existen en sus tablas padre, y detectando los casos en los que no ocurre así -un paso imprescindible antes de combinar las tablas mediante uniones (merge), ya que unir sobre claves inconsistentes habría introducido errores silenciosos en todo el análisis posterior.

4. Calidad de datos

Antes de limpiar nada, se realizó una revisión completa de la calidad del dataset siguiendo siete ejes: tipos de datos, valores nulos, duplicados, identificadores, claves foráneas, valores inconsistentes y valores atípicos (outliers). Este orden no es casual: entender primero qué problemas existen y por qué, antes de decidir cómo tratarlos, evita tomar decisiones de limpieza apresuradas o incoherentes entre sí.

4.1 Tipos de datos

Se detectaron varias variables cuyo tipo almacenado no se correspondía con su naturaleza real: marcas temporales guardadas como enteros o decimales (original_listed_time, listed_time, expiry, closed_time, time_recorded), identificadores guardados como float64 en lugar de como enteros o texto (company_id, zip_code, fips), y variables binarias o categóricas codificadas como numéricas (remote_allowed, sponsored, inferred). Se comprobó, por ejemplo, que original_listed_time, listed_time y expiry contienen valores del orden de $1,7 \times 10^{12}$, coherentes con timestamps Unix en milisegundos, mientras que time_recorded está en segundos. Estas variables se corrigieron a su tipo adecuado (datetime, entero nullable o string según el caso) en la fase de limpieza.

4.2 Valores nulos

El análisis de nulos reveló que el problema de calidad más importante del dataset se concentra en las variables económicas de postings: hasta un 94,93% de nulos en med_salary y un 75,94% en min_salary y max_salary. Esto se explica porque solo 36.073 de las 123.849 ofertas tienen un registro correspondiente en la tabla salaries; el resto simplemente no declaró información salarial en el momento de la publicación. También se detectaron 4.712 registros de salaries cuyo job_id no existe en postings, lo que confirma que la información salarial nunca puede considerarse universal. En variables categóricas como formatted_experience_level (23,75% de nulos) se decidió no descartar los registros, sino tratarlos como una categoría explícita («Unknown»), ya que representan información realmente desconocida y no un error.

4.3 Duplicados

No se encontraron filas duplicadas en ninguna de las once tablas, y las claves identificadoras principales (job_id en postings y salaries, company_id en companies) resultaron únicas, por lo que no fue necesario eliminar registros por este motivo.

4.4 Identificadores

Se revisaron los seis identificadores presentes en el dataset (company_id, job_id, salary_id, industry_id, skill_abr y sus combinaciones en las tablas de relación), confirmando que los identificadores principales de cada tabla no presentan nulos ni duplicados, y que las claves compuestas de las tablas relacionales no generan combinaciones repetidas.

4.5 Claves foráneas

Se comprobó, tabla a tabla, que los identificadores que aparecen en una tabla hija existen realmente en su tabla padre correspondiente. El resultado más relevante fue la existencia de IDs huérfanos en varias tablas relacionadas con postings:

Tabla hija	Clave foránea → padre	IDs huérfanos detectados
salaries	job_id → postings ▾	4.712
benefits	job_id → postings ▾	2.550
jobs_industries	job_id → postings ▾	5.845
jobs_skills	job_id → postings ▾	7.990

Tabla 2. IDs huérfanos detectados por tabla (job_id sin oferta asociada en postings).

En cambio, todos los industry_id, skill_abr y company_id de las tablas de referencia y relación se verificaron correctamente contra sus tablas padre. Decisión tomada: los job_id huérfanos se descartan de forma natural al realizar las uniones (merge) con postings, puesto que no existe ninguna oferta sobre la que asociar esa información.

4.6 Valores inconsistentes

Se revisaron todas las variables categóricas del dataset en busca de valores que

representen lo mismo escrito de forma distinta, o valores sin sentido dentro de su variable. La mayoría de las categóricas (formatted_work_type, work_type, formatted_experience_level, application_type, pay_period, currency, industry, skill_abr...) resultaron coherentes y sin problemas relevantes. La excepción más importante fue la variable state de companies, con una inconsistencia clara entre nombres completos y abreviaturas oficiales (California / CA, Texas / TX, New York / NY), además de un valor 0 sin significado válido. También se detectaron posibles duplicados conceptuales en company_specialities.speciality (por ejemplo, Recruiting y Recruitment), marcados para revisión en la fase de normalización.

4.7 Outliers

El análisis de valores atípicos se realizó mediante el método del rango intercuartílico (IQR) sobre las variables numéricas con sentido para este análisis (se descartaron identificadores, códigos geográficos y variables temporales codificadas como números). Los resultados principales fueron:

Variable	Outliers detectados	% sobre datos disponibles
views	16.902	13,84%
applies	2.897	12,42%
annual_med_salary	-	8,94%
annual_max_salary	-	4,34%
annual_min_salary	-	2,22%
employee_count	-	15,14% (por encima de 7.278,5 empleados)
follower_count	-	14,54% (por encima de 181.216,75 seguidores)

Tabla 3. Resumen de outliers detectados mediante el método IQR.

Antes de comparar salarios expresados en distintas periodicidades (anual, mensual, semanal, quincenal, horaria), se normalizaron todos a valor anual con los factores de

conversión estándar ($\times 1$, $\times 12$, $\times 52$, $\times 26$ y $\times 2.080$ respectivamente). Tras esta normalización se detectaron valores extremos muy elevados -salarios anuales de hasta 500-575 millones-, así como salarios máximos declarados de 120 millones de dólares. La decisión, justificada en el apartado 5.7, fue no eliminar estos valores de forma automática: pueden corresponder a puestos ejecutivos o directivos reales, y eliminarlos sin evidencia suficiente supondría perder información potencialmente válida.

5. Limpieza de datos

A partir del diagnóstico de calidad, se aplicó un proceso de limpieza estructurado en siete pasos, siempre documentando la justificación de cada decisión.

5.1 Corrección de tipos

Se aplicaron las correcciones identificadas en el apartado 4.1: conversión de las marcas temporales a `datetime`, `company_id` a entero nullable, `zip_code` y `fips` a `string` (para preservar ceros a la izquierda) y `benefits.inferred` a booleano.

5.2 Tratamiento de valores nulos

La decisión general adoptada fue no imputar valores nulos en variables donde no existe información suficiente para determinar de forma fiable cuál debería ser el valor real, especialmente en las variables salariales: inventar un salario allí donde no se declaró habría introducido sesgo artificial en todo el análisis posterior. La única excepción notable es `formatted_experience_level`, donde el nulo sí se transformó en una categoría explícita (`Unknown`), porque en este caso el propio hecho de no saber el nivel de experiencia es una información categórica válida y no imputable como número. Se muestra a continuación un resumen de las decisiones más relevantes:

Variable	% nulos	Decisión	Justificación
med_salary / max_salary / min_salary	75-95%	Mantener NaN	No se puede inventar un salario
remote_allowed	87,69%	Mantener NaN	NaN no equivale necesariamente a "no remoto"

Variable	% nulos	Decisión	Justificación
formatted_experience_level	23,75%	Categoría "Unknown" ▾	La ausencia es en sí misma información categórica
company_size (companies)	11,33%	Mantener NaN ▾	Categorías conocidas, sin base para imputar
description (companies)	1,21%	Imputar ("Sin descripció... ▾	Muy pocos nulos, impacto mínimo
description (postings)	0,006%	Rellenar ▾	Solo 7 registros afectados

Tabla 4. Resumen de las principales decisiones de tratamiento de nulos.

5.3 Tratamiento de duplicados

Al no haberse detectado duplicados en el apartado 4.3, no fue necesaria ninguna acción adicional en esta fase.

5.4 Normalización de categorías

El esfuerzo principal de esta fase se concentró en la variable state de companies, que presentaba 789 valores distintos debido al uso simultáneo de nombres completos y abreviaturas (California / CA), diferencias de formato y capitalización, errores ortográficos, valores que en realidad correspondían a ciudades o direcciones, códigos numéricos sin sentido y el valor no válido 0. Se unificaron todas estas variantes en las abreviaturas oficiales de estado, revisando los casos ambiguos con ayuda de la información disponible en city y country. Un tratamiento similar, aunque de menor escala, se aplicó a country: de las 81 categorías detectadas, se identificaron los valores no válidos 0 y OO, que se convirtieron a NaN al no existir base suficiente para inferir el país real. El resto de variables categóricas del dataset (formatted_work_type, work_type, formatted_experience_level, pay_period, currency, industry, skill_abr, entre otras) se revisaron y se mantuvieron sin cambios al no presentar inconsistencias relevantes.

5.5 Tratamiento de texto

Se revisaron las principales variables de texto (name, description, city, address, url, entre otras) en busca de valores nulos o cadenas vacías encubiertas. Las acciones más relevantes fueron sustituir los valores 0 de zip_code y los guiones (-) de address por

NaN, y eliminar espacios sobrantes al inicio y final de title mediante `.str.strip()`. El resto de variables de texto, incluida la descripción completa de cada oferta, se mantuvieron sin modificaciones semánticas para preservar la información original.

5.6 Tratamiento de salarios

Se detectaron 16 registros con salario medio igual a cero, tratados como valores no válidos y convertidos a NaN (un salario de 0 representa ausencia de información, no un salario real). Además, al anualizar los salarios expresados en distintas periodicidades, se identificaron 42 registros con importes anualizados superiores a 1.000.000 USD procedentes de periodicidades HORARY, quincenal o mensual, cuyos valores resultaban incoherentes con la unidad de pago declarada; estos se trataron como valores no plausibles. Los salarios anuales elevados registrados directamente como periodicidad YEARLY, en cambio, se conservaron al no existir evidencia suficiente para considerarlos erróneos.

5.7 Outliers

La decisión final, tras el análisis del apartado 4.7, fue conservar los valores atípicos detectados en views, applies, employee_count, follower_count y en las variables salariales, sin eliminarlos ni modificarlos. El razonamiento es el mismo en todos los casos: estos valores pueden representar situaciones reales (una oferta con mucha visibilidad, una empresa muy grande, un salario directivo elevado) y no existe evidencia suficiente de que se trate de errores de carga. Eliminarlos automáticamente habría supuesto perder información potencialmente válida y sesgar el análisis posterior. Esta decisión tiene una consecuencia directa que se retoma en la Parte II de esta memoria: al no eliminar los outliers salariales, el modelo de Machine Learning entrenado sobre ellos debe lidiar con esos valores extremos, lo que explica en parte algunos de los resultados obtenidos al intentar mejorar el modelo.

6. Análisis por tipo de variable

Con el dataset ya limpio, se realizó un análisis exploratorio organizado por tipo de variable: texto, habilidades, industrias y beneficios.

6.1 Variables de texto

Las variables de texto relevantes son title, description y skills_desc. El análisis de frecuencia de palabras en title mostró que los términos más repetidos son manager (15.645 apariciones), engineer (9.135), sales (8.103) y senior (7.932), junto con términos

como assistant, specialist, nurse, analyst, developer y director, reflejando una amplia variedad de roles profesionales, de gestión y técnicos. En description, las palabras más frecuentes fueron experience, team, ability, customer, support, management, skills, years, sales, required, care, service, health, data y development, lo que indica que las ofertas dan mucho peso a la experiencia y competencias requeridas, al trabajo en equipo y la gestión, y a los perfiles orientados a cliente, ventas, salud, datos y desarrollo. Por último, skills_desc se descartó de este análisis: al tener un 98% de valores nulos, no permite extraer ningún patrón representativo del conjunto de ofertas.

6.2 Skills

Sobre el catálogo de 35 habilidades disponible en el dataset (tabla skills), la más frecuente es Information Technology, seguida de Sales y Management, con presencia también relevante de Manufacturing, Health Care Provider, Business Development y Engineering. En cuanto al número de habilidades por oferta, de las 126.807 ofertas con al menos una skill identificada, la media es de 1,69 skills por oferta (mediana 2), con un máximo observado de 3. Esto confirma que, en general, cada oferta se asocia a una o dos grandes áreas de habilidad, más que a un perfil muy transversal.

6.3 Industrias

Entre las 422 industrias del catálogo, Hospitals and Health Care destaca claramente sobre el resto con 18.326 asociaciones a ofertas, seguida de Retail, IT Services and IT Consulting, Staffing and Recruiting y Financial Services. Este predominio del sector sanitario en el volumen de ofertas es coherente con el peso que health y care tienen entre las palabras más frecuentes de description.

6.4 Beneficios

El beneficio más frecuente con diferencia es el plan de pensiones 401(k), presente en 24.231 ofertas, seguido de distintos tipos de seguro: médico (9.873), de visión (9.309), de invalidez (7.930) y dental (6.868). En conjunto, se observa un claro predominio de los beneficios relacionados con la jubilación y los seguros médicos sobre otro tipo de compensación no salarial.

7. Resumen de la fase de exploración y limpieza

Esta primera fase del proyecto partió de once tablas relacionadas, con la tabla postings (123.849 ofertas) como eje central, y siguió un proceso estructurado en tres etapas:

exploración y diagnóstico de calidad, limpieza de datos, y análisis descriptivo por tipo de variable.

El diagnóstico de calidad identificó los problemas principales del dataset: variables temporales almacenadas como valores numéricos, identificadores mal tipados, una alta proporción de nulos en las variables salariales (entre el 70% y el 95% según la variable, derivada de que solo 36.073 de las 123.849 ofertas tienen información salarial asociada), inconsistencias de formato en variables geográficas (especialmente state) y valores atípicos relevantes en visualizaciones, solicitudes, tamaño de empresa, seguidores y salarios. No se detectaron filas duplicadas en ninguna tabla.

Las decisiones de limpieza se guiaron por un mismo criterio en todos los casos: no inventar información donde no existe evidencia suficiente para hacerlo. Esto llevó a mantener como valores nulos la inmensa mayoría de los datos ausentes (en lugar de imputarlos), a tratar la ausencia de nivel de experiencia como una categoría explícita en vez de descartarla, y a conservar los valores atípicos detectados en variables numéricas -incluidos los salarios extremos-en lugar de eliminarlos automáticamente, por considerarse potencialmente reales.

Con el foco puesto en el mercado estadounidense -que representa 116.791 de las 123.849 ofertas del dataset, un 94,30% del total-el análisis descriptivo confirmó patrones coherentes con el sentido común del mercado laboral: predominio de roles de gestión, ventas e ingeniería en los títulos de las ofertas; fuerte peso del sector sanitario en el volumen de industrias; habilidades técnicas como Information Technology y Sales como las más solicitadas; y una compensación no salarial centrada mayoritariamente en planes de pensiones y seguros médicos.

Con este trabajo, el dataset quedó preparado -en calidad, en tipos y en cobertura-para la segunda fase del proyecto: el análisis de relación entre variables y salario, y la construcción de un modelo predictivo, que se desarrolla en la Parte II de esta memoria.

PARTE II

Modelado predictivo, NLP, Spark y despliegue

Esta segunda parte de la memoria recoge el trabajo realizado a partir del dataset ya limpio y explorado en la Parte I: el análisis de qué variables se relacionan realmente con el salario, la incorporación de una fuente de datos externa obligatoria, la extracción de habilidades técnicas mediante NLP, el entrenamiento y evaluación de un modelo de Machine Learning, la exploración de mejoras sobre ese modelo, su verificación cruzada con Apache Spark, y su despliegue como servicio mediante una API.

8. Correlación

El primer paso antes de decidir qué variables incorporar al modelo fue comprobar cuáles tienen relación real con el salario. Se calculó la correlación entre `normalized_salary` y las variables numéricas disponibles directamente en el dataset: `views` y `applies`. El resultado fue prácticamente nulo en ambos casos ($r = -0,0016$ con `views` y $r = -0,0023$ con `applies`), lo que indica que el salario de una oferta no influye en cuántas veces se visualiza ni en cuántas solicitudes recibe. La única correlación relevante detectada fue entre `views` y `applies` ($r = 0,48$): a más visualizaciones, más solicitudes, algo esperable pero sin relación con el objetivo del proyecto. En consecuencia, ambas variables se descartaron del modelo desde este primer análisis.

También se exploró la relación entre salario y nivel de experiencia. El primer cálculo, usando la media, dio resultados sin sentido: `Internship` aparecía con una media de más de 1.000.000 \$ y `Entry level` por encima de `Mid-Senior level`. La causa es la misma que se documentó en el apartado 4.7 de la Parte I: al no eliminar los outliers salariales, unos pocos valores extremos dentro de un grupo pequeño (como `Internship`) distorsionan por completo la media de ese grupo. La solución fue recalcular con la mediana, mucho más robusta frente a valores extremos:

Nivel de experiencia	Salario mediano	Nº de ofertas
Internship	48.880 \$	361
Entry level	52.000 \$	8.427

Nivel de experiencia	Salario mediano	Nº de ofertas
Associate	72.500 \$	3.439
Unknown	77.500 \$	7.115
Mid-Senior level	105.952,5 \$	11.442
Director	165.000 \$	1.118
Executive	190.000 \$	334

Tabla 5. Salario mediano por nivel de experiencia, ordenado de menor a mayor seniority.

Con la mediana, el salario sube de forma clara y coherente según la seniority, desde 48.880 \$ en Internship hasta 190.000 \$ en Executive. Este hallazgo resultó clave más adelante: demuestra que el nivel de experiencia tiene, con diferencia, la relación más fuerte y consistente con el salario de todo el dataset, muy por encima de cualquier skill individual -un dato que se retoma en el apartado 13, al justificar por qué merecía la pena probar a incorporar esta variable al modelo.

9. Cruce del salario con el índice de coste de vida por estado (EE. UU.)

El enunciado del proyecto exige incorporar al menos una fuente de datos externa al dataset original. De entre las opciones razonables (coste de vida, tasa de paro por estado, salario mínimo por estado...), se eligió el índice de coste de vida por estado (Cost of Living Index, World Population Review / MERIC, 2026) por tener una lógica de negocio directa con el objetivo del proyecto: es un hecho conocido que las empresas ajustan salarios según lo caro que resulta vivir en cada zona, de modo que un mismo puesto suele pagar de forma distinta en un estado caro que en uno barato.

El cruce se realizó extrayendo el estado a partir de la columna location de cada oferta y uniéndolo (merge) con el fichero externo de coste de vida por estado. La cobertura resultante fue buena: 94.353 ofertas quedaron con un índice de coste de vida asignado (85,8% de las ofertas de EE. UU. con estado identificable), frente a 15.642 que no pudieron cruzarse por falta de un formato de localización reconocible.

El resultado de la correlación directa fue sorprendente: prácticamente nula ($r = 0,0017$) entre el índice de coste de vida y el salario normalizado. A primera vista, esto sugeriría que la variable no aporta nada y podría descartarse del modelo. Sin embargo, como se explica en el apartado 12, esta conclusión resultó ser incorrecta: el efecto del coste de vida sobre el salario no es lineal ni constante, sino que depende de la industria -y ese tipo de relación por grupos es invisible para una correlación simple, pero perfectamente detectable por un modelo como Random Forest. Este es, probablemente, el hallazgo metodológico más importante de todo el proyecto.

10. Extracción de habilidades técnicas desde el texto (NLP)

El dataset no incluye, de forma estructurada, las habilidades técnicas concretas que pide cada oferta más allá de la asociación genérica a 35 categorías amplias de la tabla skills. Para poder incorporar habilidades técnicas específicas (Python, SQL, AWS...) al modelo, fue necesario extraerlas del texto libre de la descripción de cada oferta (description) mediante procesamiento de lenguaje natural.

Se definió una lista de 29 habilidades relevantes en el ámbito de datos e IA: Python, SQL, R, Excel, Power BI, Tableau, AWS, Azure, GCP, Spark, Hadoop, Machine Learning, Deep Learning, TensorFlow, PyTorch, NLP, Scikit-learn, Pandas, Git, Docker, Kubernetes, Java, Scala, Airflow, ETL, Statistics, Linux, API, Snowflake y MongoDB. Cada una se buscó en el texto de la oferta como palabra completa, usando expresiones regulares con límites de palabra (word boundaries), precisamente para evitar falsos positivos -por ejemplo, que la palabra "excellent" se confundiera con la skill "Excel". El resultado se validó revisando manualmente una muestra de ofertas marcadas con la skill "SQL", confirmando que las coincidencias detectadas eran correctas en todos los casos revisados.

El resultado de esta extracción fue la tabla skills_df, con una fila por oferta y una columna binaria (0/1) por cada una de las 29 skills. Las habilidades más solicitadas en el texto de las ofertas resultaron ser:

Skill	Menciones	Skill	Menciones
Excel	16.232	Tableau	1.120
SQL	4.624	Git / Kubernetes	1.016

Skill	Menciones	Skill	Menciones
R	4.323	Power BI	923
Python	4.022	Docker	911
AWS	2.783	GCP	866
Azure	2.567	ETL	774
Java	2.332	Spark	727
Statistics	1.769	Machine Learning	1.490

Tabla 6. Skills técnicas más mencionadas en el texto de las ofertas (top 16 de 29).

Esta tabla de skills es una de las tres piezas centrales que, junto con la industria y el índice de coste de vida, conforman las variables de entrada del modelo de predicción de salario descrito en el apartado 12.

11. Preparar la tabla para el modelo de salario

Antes de entrenar cualquier modelo, fue necesario construir una única tabla que combinara, para cada oferta, sus 29 skills (0/1), su industria y su índice de coste de vida, junto con el salario a predecir. El proceso siguió varios pasos de unión y filtrado, cada uno con su correspondiente pérdida controlada de filas:

- Asignación de industria: de las 109.995 ofertas de EE. UU., 109.859 quedaron con una industria asignada (se tomó la primera cuando una oferta tenía más de una asociada, para simplificar), y solo 136 quedaron sin industria.
- Unión de las 29 columnas de skills a la tabla principal, sin pérdida de filas (109.995 filas, 66 columnas resultantes).
- Filtro de ofertas con salario conocido (normalized_salary no nulo): de las 109.995 ofertas, 32.236 tenían salario informado; las 77.759 restantes se descartaron, ya que sin salario no pueden usarse ni para entrenar ni para evaluar el modelo.
- Construcción de X (variables de entrada: 29 skills + índice de coste de vida + industria codificada como columnas 0/1) e y (salario a predecir). La forma final de

X fue (32.236, 363): 363 columnas, resultado de 29 skills + 1 de coste de vida + en torno a 330 columnas de industria.

- Eliminación de nulos: 5.308 filas se perdieron porque su índice de coste de vida no pudo asignarse (ofertas cuya location no seguía un formato reconocible), quedando X e y alineados con 26.928 filas.
- Filtro final de salarios inválidos: se descartaron 65 filas adicionales con salario igual a 0 \$ o superior a 500.000 \$ anuales, quedando la tabla definitiva en 26.863 ofertas.

Con este proceso, la tabla final para entrenar el modelo queda compuesta por 26.863 ofertas, cada una descrita por 363 variables (29 skills técnicas, el índice de coste de vida de su estado y su industria codificada), con el salario normalizado como variable objetivo.

12. Entrenar el modelo de salario (Random Forest)

Con la tabla final ya preparada, se dividió en un conjunto de entrenamiento (80%, 21.490 ofertas) y un conjunto de prueba (20%) mediante `train_test_split` con `random_state=42` para que el experimento fuera reproducible. Se entrenó un modelo Random Forest Regressor con 100 árboles de decisión (`n_estimators=100`). Un Random Forest construye muchos árboles de decisión distintos (en este caso, 100), cada uno entrenado con una muestra y una selección de variables ligeramente distinta, y combina sus predicciones mediante la media -de forma similar a preguntar a 100 personas por separado y quedarse con el promedio de sus respuestas, en lugar de fiarse de una única opinión.

Evaluado sobre el conjunto de prueba (ofertas que el modelo no vio durante el entrenamiento), el resultado fue:

Métrica	Valor	Interpretación
MAE (Error Absoluto Medio)	34.691,49 \$	De media, el modelo se equivoca en unos 34.700 \$ al predecir el salario de una oferta
R ²	0,191 (19,1%)	Las variables usadas explican cerca del 19% de la variación del salario entre ofertas

Tabla 7. Métricas del modelo base (Random Forest, skills + industria + coste de vida).

Un R^2 de 0,191 significa que el 81% restante de la variación del salario depende de factores no incluidos en el modelo, principalmente el nivel de experiencia, que en el análisis descriptivo del apartado 8 ya mostró una relación mucho más fuerte y consistente con el salario que cualquier skill individual. Este resultado, aunque modesto en términos absolutos, confirma que el modelo capta una parte real y útil del patrón salarial usando únicamente skills, industria y coste de vida.

12.1 Importancia de las variables

Analizar qué variables usa más el modelo para predecir permite entender su comportamiento interno más allá de la métrica global:

Variable	Importancia	Variable	Importancia
indice_coste_vida	28,7%	AWS	2,5%
industria_Retail	5,5%	R	2,2%
Excel	5,2%	Java	1,9%
industria_Software Development	4,2%	SQL	1,4%
Machine Learning	4,0%	Statistics	1,2%
Python	3,7%	industria_IT Services and IT Consulting	1,2%

Tabla 8. Top 12 de variables más importantes del modelo base.

El resultado más llamativo, y el hallazgo central de este proyecto, es que el índice de coste de vida -una única variable, con una correlación directa con el salario prácticamente nula ($r = 0,0017$, ver apartado 9)-resulta ser, por sí sola, el predictor individual más importante del modelo, muy por encima de cualquier skill o industria concreta. Agrupando la importancia por tipo de variable, la industria pesa algo más en

conjunto (41,6%, repartida entre unas 330 columnas) que las skills (29,7%, repartida entre 29 columnas) y que la región/coste de vida (28,7%, una única variable).

La explicación de esta aparente contradicción -correlación casi nula pero máxima importancia-es la clave metodológica de todo el proyecto: la correlación solo detecta relaciones lineales y constantes (si sube una variable, sube o baja la otra siempre de la misma forma). El efecto real del coste de vida sobre el salario, sin embargo, depende de la industria: probablemente en sectores como el tecnológico, vivir en una zona cara sí eleva claramente el salario ofrecido, mientras que en otros sectores apenas influye. Si se mezclan todas las industrias juntas, como hace un cálculo de correlación simple, esos efectos opuestos se cancelan entre sí y el resultado parece nulo. Un modelo como Random Forest, en cambio, sí es capaz de detectar relaciones de este tipo, condicionadas por otras variables, y por eso identifica el coste de vida como determinante. Esto confirma que el cruce con la fuente de datos externa (apartado 9) fue una decisión acertada, y demuestra por qué no basta con mirar la correlación simple para decidir qué variables incluir en un modelo.

12.2 Función de predicción y exportación del modelo

Se implementó una función `predecir_salario(skills_seleccionadas, industria, indice_coste_vida_valor)` que reconstruye, a partir de estos tres inputs, la misma fila de variables que espera el modelo entrenado, y devuelve el salario estimado. La función incorpora avisos de validación: si alguna skill o industria proporcionada no coincide con las que conoce el modelo, se informa explícitamente de que ese dato se ignorará en la predicción, en lugar de fallar silenciosamente. A modo de ejemplo, para una oferta con las skills Python, SQL y Machine Learning, industria Software Development e índice de coste de vida 142,3, el modelo estima un salario de 148.082,6 \$.

Finalmente, el modelo entrenado (`modelo_salario.pkl`) y la lista de las 363 columnas que espera como entrada (`columnas_modelo.pkl`) se exportaron mediante `joblib`, quedando listos para integrarse en el servicio de predicción descrito en el apartado 15.

13. Mejora del modelo: logaritmo del salario, Gradient Boosting y nivel de experiencia

Con el modelo base ya entrenado, evaluado y desplegado, se dedicó una semana adicional -antes de la entrega final-a explorar de forma rigurosa si el resultado podía mejorarse, probando tres vías distintas y documentando el resultado de cada una,

funcionara o no. Como medida de seguridad, antes de empezar se realizó una copia de respaldo de los ficheros del modelo ya exportado (modelo_salario.pkl y columnas_modelo.pkl), de forma que ninguna de las pruebas pusiera en riesgo la versión ya validada y desplegada en el endpoint.

13.1 Logaritmo del salario

La distribución del salario está sesgada hacia la derecha: la mayoría de las ofertas se agrupan en un rango razonable, pero existe una minoría de sueldos muy altos (puestos ejecutivos y directivos) que se alejan mucho del resto. Como se decidió no eliminar esos outliers salariales en la fase de limpieza (apartado 5.7), el modelo entrenado sobre el salario en bruto tiene que intentar ajustarse también a esos valores extremos, lo que puede perjudicar su precisión en el rango donde se concentra la mayoría de las ofertas. Aplicar logaritmo al salario, mediante $\log(1 + \text{salario})$, comprime esa escala: los sueldos muy altos dejan de pesar tanto durante el entrenamiento, y la distribución resultante se aproxima más a una normal, que es el escenario donde este tipo de modelos suele aprender mejor.

Resultado obtenido, tras confirmar que se mantenía de forma estable tras reiniciar el kernel y volver a ejecutar el notebook completo:

Modelo			MAE (salario real)	R ²
Random Forest (salario original)			34.691,49 \$	0,191
Random Forest (log del salario)			34.941,99 \$	-0,013

Tabla 9. Comparación entre el modelo original y la versión con logaritmo del salario.

El resultado con logaritmo no solo no mejora, sino que empeora claramente, con un R² prácticamente nulo (incluso ligeramente negativo). La explicación es reveladora: en la escala original, buena parte del R²=0,191 del modelo base se debía a acertar en los pocos sueldos extremadamente altos que se decidió no eliminar como outliers -como el R² se calcula sobre el error al cuadrado, acertar en esos pocos casos gigantescos aporta muchos puntos aunque sean solo un puñado de ofertas. Al aplicar el logaritmo, esos

sueldos extremos se comprimen y dejan de pesar tanto, de modo que lo que queda por explicar es la diferencia entre salarios «normales» -y ahí, usando solo skills, industria y coste de vida, el modelo apenas tiene señal real que aprovechar. Conclusión: se descarta el logaritmo y se mantiene el modelo original. Este resultado negativo no se considera un fallo, sino un hallazgo honesto y valioso: revela que el modelo original dependía más de lo que parecía de esos pocos casos extremos.

13.2 Comparación con Gradient Boosting

Hasta este punto solo se había utilizado Random Forest, sin comparar frente a ningún algoritmo alternativo. Gradient Boosting construye los árboles de decisión de forma secuencial -cada árbol nuevo se centra específicamente en corregir los errores del árbol anterior-, en lugar de construirlos todos por separado y promediar como hace Random Forest; suele dar mejores resultados en datos tabulares como los de este proyecto. Se entrenó un HistGradientBoostingRegressor (que incorpora valores por defecto pensados para evitar sobreajuste) con las mismas variables que el modelo original, sin logaritmo, para poder comparar en igualdad de condiciones:

Modelo	MAE	R ²
Random Forest	34.691,49 \$	0,191
Gradient Boosting	34.514,49 \$	0,218

Tabla 10. Comparación entre Random Forest y Gradient Boosting.

Gradient Boosting mejora el resultado de forma moderada pero real, en la dirección esperada: al corregir los árboles de forma secuencial en vez de promediarlos por separado, consigue reducir ligeramente el error y aumentar el R². Dado que la mejora es moderada y no justifica el coste de sustituir el modelo ya validado y desplegado en el endpoint, se documenta como una alternativa viable pero no se sustituye el modelo en producción.

13.3 Incorporación del nivel de experiencia (experimento adicional)

El análisis descriptivo del apartado 8 mostró que el salario sube de forma muy clara según el nivel de experiencia (de 48.880 \$ en Internship a 190.000 \$ en Executive), una relación mucho más marcada que la de cualquier skill individual. Sin embargo, esta

variable nunca se había incluido en el modelo de predicción (X solo contenía skills, industria y coste de vida). Dado que la incorporación de una variable nueva no es una modificación menor -afecta a las columnas que espera el modelo, y por tanto también al endpoint que lo sirve-, se probó como un experimento aparte, sin tocar el modelo oficial ya exportado: se construyó una tabla nueva (X_exp) que añade las columnas de experiencia (codificadas como 0/1) a las variables ya existentes, y se entrenó un modelo nuevo únicamente para comparar, sin sustituir modelo_salario.pkl ni columnas_modelo.pkl.

Modelo			MAE	R ²
Random experiencia)	Forest -modelo	(sin en producción	34.691,49 \$	0,191
Random experiencia)	Forest -experimento	(con	30.006,80 \$	0,340

Tabla 11. Comparación entre el modelo en producción y el experimento con nivel de experiencia.

El resultado es, con diferencia, la mejora más grande de las tres exploradas: el R² sube de 0,191 a 0,340 (casi el doble) y el MAE baja de 34.691,49 \$ a 30.006,80 \$ (unos 4.685 \$ menos de error medio). En el top de variables más importantes de este modelo experimental, varias categorías de experiencia (Entry level, Director, Executive, Mid-Senior level, Unknown) aparecen entre las más influyentes, y sumadas superan incluso al índice de coste de vida, que sigue siendo la variable individual más importante (20,1%):

Variable	Importancia	Variable	Importancia
indice_coste_vida	20,1%	industria_Retail	3,2%
experiencia_Entry level	13,0%	Machine Learning	2,5%
experiencia_Director	6,2%	Python	2,4%

Variable	Importancia	Variable	Importancia
experiencia_Executive	3,5%	industria_Software Development	2,2%
experiencia_Mid-Senior level	3,4%	R / Java	1,5% / 1,4%
Excel	3,3%	experiencia_Unknown	1,3%

Tabla 12. Top de variables más importantes del modelo experimental con nivel de experiencia.

Este resultado confirma cuantitativamente lo que ya sugería el análisis descriptivo: el nivel de experiencia es, con diferencia, el factor individual que más explica el salario en este dataset, por encima incluso del coste de vida. Ahora bien, dado que incorporar esta variable de forma definitiva implicaría volver a exportar el modelo, actualizar el endpoint (endpoint.py) para que reciba y procese también el nivel de experiencia, y reconstruir la imagen de Docker verificando que la predicción de prueba sigue siendo coherente, se decidió no aplicar este cambio a última hora sobre un sistema ya funcionando y validado. El modelo en producción integrado en el endpoint sigue siendo el original (sin experiencia); esta mejora queda documentada como la línea de trabajo más clara y ya cuantificada para una próxima iteración del proyecto.

14. ETL y verificación de métricas con Apache Spark

Como parte del requisito de Big Data del proyecto, se replicó una de las métricas descriptivas obtenidas con pandas utilizando Apache Spark, con el fin de comprobar que el resultado se mantiene al procesar los datos con una herramienta de procesamiento distribuido, pensada para volúmenes de datos mayores de los que caben en memoria en una sola máquina.

Se inicializó una sesión de Spark local (SparkSession, modo local[*], aprovechando todos los núcleos disponibles) y se cargó el fichero de salarios y experiencia mediante spark.read.csv, con inferencia automática de esquema. Sobre esta tabla se calculó, usando las funciones nativas de Spark (groupBy + percentile_approx), el salario mediano por nivel de experiencia -la misma métrica ya calculada con pandas en el apartado 8.

El resultado obtenido con Spark coincidió con el calculado previamente con pandas, confirmando que ambas herramientas producen el mismo resultado sobre los mismos datos. Esta coincidencia es una validación real, no trivial: demuestra que el cálculo no depende de una particularidad de la implementación de pandas, y da confianza en que la métrica (el salario mediano subiendo de forma coherente con la seniority) es un patrón robusto de los datos y no un artefacto de la herramienta utilizada.

15. Arquitectura de despliegue: API

El modelo entrenado no se quedó únicamente en el notebook: se empaquetó como un servicio real, accesible mediante una API HTTP.

15.1 API con FastAPI

El servicio se implementó con FastAPI (fichero `api/endpoint.py`), y carga en el arranque el modelo entrenado (`modelo_salario.pkl`) y la lista de columnas que espera (`columnas_modelo.pkl`), ambos generados en el notebook de análisis. Expone dos rutas:

- GET / -comprueba que el servicio está activo y responde correctamente.
- POST /predecir -recibe una oferta de trabajo (una lista de skills, la industria y el índice de coste de vida de la región) mediante un modelo de datos validado con Pydantic (`OfertaEntrada`), reconstruye internamente la misma fila de variables que espera el modelo (reproduciendo la lógica de la función `predecir_salario()` del notebook) y devuelve el salario estimado en formato JSON.

La validación de entrada mediante Pydantic garantiza que una petición mal formada (por ejemplo, sin el campo `industria`) se rechace de forma clara antes de llegar al modelo, en lugar de fallar de forma confusa dentro de la lógica de predicción.

16. Conclusiones generales del proyecto

Resumen del trabajo realizado

Partiendo de un dataset relacional de más de 120.000 ofertas de empleo, el proyecto cubre de principio a fin un pipeline completo de ciencia de datos: exploración y diagnóstico de calidad de once tablas, limpieza y normalización de datos con decisiones justificadas en cada paso (incluida la decisión, explícita y mantenida, de no eliminar valores atípicos salvo evidencia clara de error), extracción de 29 habilidades técnicas mediante procesamiento de lenguaje natural, cruce con una fuente de datos externa obligatoria (índice de coste de vida por estado), entrenamiento y evaluación de un

modelo Random Forest para predecir el salario de una oferta, exploración rigurosa de tres vías de mejora del modelo, verificación cruzada de una métrica con Apache Spark, y despliegue del modelo como un servicio real mediante una API con FastAPI.

Principal hallazgo

El índice de coste de vida, pese a tener una correlación directa con el salario prácticamente nula ($r = 0,0017$), resultó ser la variable individual más importante del modelo base (28,7% de importancia). La explicación es que su efecto real depende de la industria: en unos sectores el coste de vida de la zona influye claramente en el salario ofrecido, y en otros apenas lo hace, de modo que al mezclar todas las industrias en un único cálculo de correlación esos efectos se cancelan entre sí. Este resultado es un ejemplo claro de por qué la correlación simple no es suficiente para decidir qué variables incluir en un modelo, y de por qué merece la pena entrenar y evaluar en lugar de descartar variables únicamente por su correlación directa con el objetivo.

Rendimiento del modelo y mejoras probadas

El modelo base (Random Forest, el que está integrado en el endpoint) explica un 19,1% de la variación del salario ($R^2 = 0,191$, MAE = 34.691,49 \$). Se probaron tres vías de mejora, documentando el resultado de cada una con independencia de si funcionaba o no: aplicar logaritmo al salario, que empeoró notablemente el resultado ($R^2 = -0,013$) y reveló que buena parte del R^2 original dependía de acertar en unos pocos sueldos extremos, no de explicar bien las diferencias salariales típicas; comparar con Gradient Boosting, que mejoró de forma moderada y en la dirección esperada ($R^2 = 0,218$, MAE = 34.514,49 \$); y añadir el nivel de experiencia como variable adicional, que fue con diferencia la mejora más grande de las tres: el R^2 sube a 0,340 (casi el doble que el modelo base) y el MAE baja a 30.006,80 \$. En este último modelo, las categorías de experiencia, sumadas, superan incluso en importancia al índice de coste de vida. El modelo en producción integrado en el endpoint sigue siendo el original, sin nivel de experiencia, por lo que esta mejora queda documentada como la línea de trabajo futura.

Limitaciones

Un R^2 entre 0,19 y 0,34, según la versión del modelo, sigue siendo un resultado modesto en términos absolutos: el salario real de una oferta depende de factores que no están presentes en estos datos, como la negociación individual de cada candidato, el tamaño exacto de la empresa más allá de las categorías disponibles, o la ciudad concreta en lugar del estado como unidad geográfica.

Cierre

El proyecto entrega un pipeline funcional y verificado de principio a fin: desde datos crudos y con múltiples problemas de calidad, hasta una predicción de salario servible mediante una API real, pasando por decisiones documentadas y validadas en cada paso -incluidas las que no funcionaron, como el logaritmo del salario, y las que sí funcionaron mucho mejor de lo esperado, como añadir el nivel de experiencia. No se trata de un modelo perfecto -ningún modelo que prediga salarios a partir de datos públicos de ofertas de empleo lo es-, sino de un trabajo riguroso sobre datos reales, con las decisiones y sus motivos explicados en cada fase, y con las limitaciones señaladas de forma explícita como líneas de trabajo futuro claramente identificadas.

17. Uso de IA

En este informe se ha hecho uso de la herramienta de Inteligencia Artificial Google Gemini para rediseñar este documento: fuente, colores. El equipo cuenta con la versión original de este archivo con otro formato, además de notas en el chat de discord resumiendo cada una de nuestras reuniones durante el análisis de datos, construcción de modelos, validación, resolución de errores, y demás.